

project introduction

بازیابی و درک تصویر با یادگیری عمیق

در طول این ترم، با مفاهیم پایه‌ای پردازش تصویر، فیلترهای مکانی، تحلیل در حوزه فرکانس و روش‌های کلاسیک برای بهبود کیفیت تصاویر آشنا شدید. اکنون در این پروژه پایانی، قصد داریم این مفاهیم را یک قدم جلوتر برده و وارد دنیای مدرن **یادگیری عمیق (Deep Learning)** شویم.

این پروژه به گونه‌ای طراحی شده است که شما را با یک سناریوی کاملاً واقعی در مهندسی بینایی ماشین روبرو کند: دریافت یک سیگنال به شدت مخدوش و تلاش برای بازیابی اطلاعات معنایی آن با استفاده از شبکه‌های عصبی پیچشی (CNN).

هدف اصلی پروژه

شما در این پروژه یک خط لوله (Pipeline) چندمرحله‌ای خواهید ساخت. ورودی سیستم شما یک تصویر به شدت مخدوش است که همزمان دچار دو مشکل اساسی شده است:

۱. **نویز فلغل‌نمکی (Salt-and-Pepper Noise):** پیکسل‌های کاملاً سفید و سیاه که اطلاعات محلی را از بین برده‌اند.

۲. **تاری (Blur):** از دست رفتن لبه‌های تیز و فرکانس‌های بالا.

هدف شما این است که این تصویر را مرحله‌به‌مرحله از شبکه‌های عصبی از پیش آموزش‌دیده عبور دهید تا نویز و تاری آن برطرف شود و در نهایت، تصویر بازیابی‌شده را به یک شبکه دسته‌بند (Classifier) بدهید تا مشخص شود آیا الگوریتم‌های شما توانسته‌اند ماهیت اصلی تصویر را به قدری خوب بازیابی کنند که هوش مصنوعی بتواند محتوای آن را تشخیص دهد یا خیر.

نقشه‌راه و مراحل انجام پروژه

این پروژه در قالب فازهای زیر انجام خواهد شد:

- **فاز صفر (آماده‌سازی محیط):** در این بخش، راهنمای کاملی برای نصب کتابخانه‌های مورد نیاز (مانند TensorFlow، PyTorch و OpenCV) و نحوه استفاده از محیط‌های پردازش ابری (Google Colab) در اختیار شما قرار می‌گیرد.
- **فاز اول (حذف نویز - Denoising):** تصویر مخدوش را دریافت کرده و با استفاده از یک مدل یادگیری عمیق، نویزهای نقطه‌ای را حذف می‌کنید تا به یک تصویر تار اما بدون نویز برسید.
- **فاز دوم (رفع تاری - Deblurring و ارزیابی کمی):** خروجی فاز قبل را به مدل‌های رفع تاری می‌دهید. در این بخش، چندین معماری مختلف شبکه عصبی در اختیار شما قرار می‌گیرد. شما باید خروجی این مدل‌ها را با استفاده از معیار ریاضی **PSNR** با تصویر اصلی (بدون مخدوش‌شدگی) مقایسه کرده و بهترین مدل را انتخاب کنید.
- **فاز سوم (دسته‌بندی - Classification):** در گام نهایی، یک شبکه عصبی را روی مجموعه داده استاندارد آموزش می‌دهید و تصویر بازیابی‌شده خود را به آن می‌سپارید تا کلاس تصویر (مثلاً ماشین، هواپیما، حیوان و...) را پیش‌بینی کند.

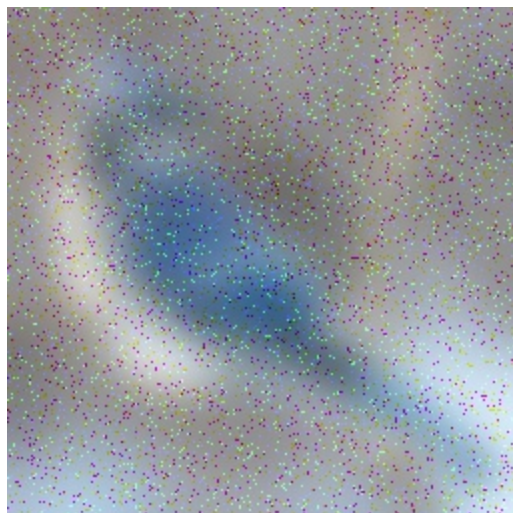
داده‌های ورودی (فایل‌های پیوست)

برای شروع پروژه، دو فایل تصویری در این صفحه برای شما قرار داده شده است. لطفاً آن‌ها را دانلود کرده و در پوشه پروژه خود ذخیره کنید:

۱. **نوت‌بوک جامع کدهای پروژه (Jupyter Notebook):** تمامی کدهای مورد نیاز برای اجرای این پروژه (شامل توابع پردازشی، معماری شبکه‌ها و ابزارهای ارزیابی) در قالب یک فایل جامع `ipynb` در اختیار شما قرار گرفته است. **شما نیازی به نوشتن کدهای شبکه عصبی از صفر ندارید.** وظیفه شما در این پروژه، درک منطق درونی این توابع و فراخوانی صحیح آن‌ها با فایل‌ها و ورودی‌های درست است.

[jupyter notebook file](#)

۲. **تصویر مخدوش (corrupted_image.jpg یا فایل مشابه):** این تصویر ورودی اصلی خط لوله شماست و تمام عملیات پردازشی روی آن انجام خواهد شد.



۳. تصویر مرجع / اصلی (original_image.jpg یا فایل مشابه): این نسخه کاملاً تمیز و دست‌نخورده از همان تصویر است. شما حق ندارید این تصویر را به شبکه‌های عصبی خود بدهید. این فایل تنها به عنوان Ground Truth در فاز دوم و برای محاسبه میزان موفقیت الگوریتم‌های شما (تابع PSNR) استفاده خواهد شد.



با دقت مراحل بعدی را در صفحات آینده دنبال کنید. موفق باشید!

installation and setup

فاز صفر: ✂ نصب و آماده‌سازی محیط توسعه

این پروژه ترکیبی از دو فریم‌ورک قدرتمند **PyTorch** و **TensorFlow** است. از آنجایی که پردازش تصویر و یادگیری عمیق به توان محاسباتی بالایی نیاز دارد، در صورتی که سیستم شما دارای کارت گرافیک (GPU) مناسب است، توصیه می‌کنیم پیش از نصب پیش‌نیازهای عمومی پروژه، محیط محلی CUDA خود را به درستی پیکربندی کنید تا بتوانید از شتاب‌دهنده گرافیکی بهره‌مند شوید.

(نکته: در صورتی که سیستم قدرتمندی ندارید، می‌توانید از پلتفرم‌های ابری مانند *Google Colab* استفاده کنید که به صورت پیش‌فرض بسیاری از این نیازمندی‌ها را در خود جای داده‌اند.)

۱. پیش‌نیازها

- **آناکوندا (Anaconda) یا مینی‌کوندا (Miniconda):** استفاده از این ابزارها برای مدیریت محیط‌های مجازی و جلوگیری از تداخل نسخه‌ها به شدت توصیه می‌شود.
- **درایورهای کارت گرافیک (اختیاری اما پیشنهادی):** اگر قصد دارید مدل‌ها را روی پردازنده گرافیکی (GPU) اجرا کنید، اطمینان حاصل کنید که درایورهای مناسب NVIDIA روی سیستم شما نصب شده باشند.

۲. ایجاد محیط مجازی (Virtual Environment)

برای جلوگیری از تداخل نیازمندی‌های این پروژه با سایر پروژه‌هایتان، یک محیط مجازی جدید در Conda ایجاد و سپس آن را فعال کنید. دستورات زیر را در ترمینال خود وارد کنید:

Bash

```
conda create -n dl_project_env python=3.10
conda activate dl_project_env
```

۳. نصب فریم‌ورک‌های یادگیری عمیق (وابسته به سخت‌افزار شما)

شما باید فریم‌ورک‌های یادگیری عمیق را دقیقاً متناسب با سخت‌افزار و ابزار CUDA نصب شده روی سیستم خودتان نصب کنید.

- **کتابخانه PyTorch:** به [صفحه نصب PyTorch](#) مراجعه کنید تا دستور نصب دقیق متناسب با سیستم‌عامل و پلتفرم پردازشی خود (CUDA / ROCm / CPU) را دریافت کنید.
- **کتابخانه TensorFlow:** برای اطمینان از همگام‌سازی درست مسیرهای cuDNN محلی، به [راهنمای نصب TensorFlow GPU](#) مراجعه نمایید.

مثال (دستور نصب PyTorch برای سیستم‌های دارای CUDA 11.8):

Bash

```
conda install pytorch torchvision pytorch-cuda=11.8 -c pytorch -c nvidia
```

۴. نصب نیازمندی‌های عمومی پروژه

پس از اینکه فریم‌ورک‌های اصلی و تنظیمات سخت‌افزاری را پیکربندی کردید، نوبت به نصب سایر کتابخانه‌های پردازش تصویر و مدیریت داده می‌رسد. فایل [requirements.txt](#) را با دستور زیر نصب کنید:

Bash

```
pip install -r requirements.txt
```

۵. ایجاد ساختار پوشه‌های پروژه

پیش از اجرای کدهای پروژه، باید ساختار دایرکتوری‌های مورد نیاز را ایجاد کنید. ترمینال خود را در مسیر اصلی (Root) پروژه باز کرده و دستور پایتون زیر را اجرا کنید تا پوشه‌های لازم به صورت خودکار ساخته شوند:

Bash

```
python -c "import os; [os.makedirs(d, exist_ok=True) for d in ['models', 'nois
```

با انجام موفقیت‌آمیز این مراحل، محیط شما برای شروع فاز اول (Denoising) کاملاً آماده است!

⚠️ توجه بسیار مهم: اگر با نصب و پیکربندی فریم‌ورک‌های یادگیری عمیق (مانند PyTorch و TensorFlow) آشنایی ندارید، یا درگیر شدن با تنظیمات کارت گرافیک، محیط‌های مجازی و درایورهای CUDA برایتان پیچیده است، برای راحتی خود و جلوگیری از اتلاف وقت، حتماً از Google Colab استفاده کنید! پلتفرم Colab یک محیط ابری کاملاً آماده با دسترسی رایگان به پردازنده گرافیکی (GPU) در اختیار شما قرار می‌دهد که تمامی این کتابخانه‌ها از پیش روی آن نصب شده‌اند و نیازی به هیچ‌گونه تنظیمات محلی ندارید.

Denoising

فاز اول: پاک‌سازی و حذف نویز عمیق (Deep Denoising)

در این مرحله، شما اولین گام عملی برای بازیابی تصویر مخدوش را برمی‌دارید. تصویر ورودی شما دارای دو مشکل اساسی است: نویز شدید فلفل‌نمکی (نقاط مزاحم سیاه و سفید) و تاری. هدف این فاز، استفاده از یک شبکه عصبی کانولوشنی (CNN) برای حذف نویزهای نقطه‌ای است، به طوری که ساختار کلی تصویر برای مرحله بعد (رفع تاری) حفظ شود.

۱. دانلود مدل از پیش آموزش‌دیده

برای این فاز نیازی به آموزش مدل از صفر ندارید. ما یک مدل قدرتمند TensorFlow/Keras را برای شما در نظر گرفته‌ایم که روی حذف نویز تصاویر آموزش دیده است.

۱. به لینک زیر در پلتفرم Kaggle مراجعه کنید:

[دانلود مدل \(Kaggle\) Denoising](#)

۲. فایل `denoising_model.h5` را دانلود کنید.

۳. این فایل را مستقیماً در پوشه `models` قرار دهید.

۲. نحوه استفاده از کد (تابع `denoising`)

در فایل نوت‌بوک (Jupyter Notebook) که در اختیار شما قرار گرفته است، تابع `denoising` به صورت کامل و آماده اجرا نوشته شده است. این تابع تمام مراحل پیچیده از جمله موارد زیر را برای شما انجام می‌دهد:

- بارگذاری مدل بدون نیاز به کامپایل مجدد (`compile=False`).
- تغییر ابعاد تصویر به 256×256 پیکسل (اندازه استاندارد مورد انتظار مدل).
- نرمال‌سازی مقادیر پیکسل‌ها به بازه $[0.0, 1.0]$ و اضافه کردن بُعد دسته (Batch Dimension) برای پردازش در شبکه عصبی.
- بازگرداندن تصویر به مقیاس استاندارد ۸ بیتی (`uint8`) پس از پردازش مدل.

وظیفه شما:

شما تنها باید این تابع را در نوت‌بوک خود با مسیرهای (Paths) درست فراخوانی کنید:

Python

```
denoising(image_path="...", dest_path="...", model_path="...")
```

۳. تنظیم مسیرها و خروجی مورد انتظار

هنگام فراخوانی تابع، به ساختار پوشه‌های پروژه دقت کنید:

- **مسیر ورودی (image_path)**: تصویر مخدوش خود را از پوشه `noisy-inputs` انتخاب کنید (مثلاً `( noisy-inputs/noisy_image_car.jpeg`
- **مسیر مدل (model_path)**: مسیر فایلی که در گام اول دانلود کردید (مثلاً `( models/denoising_model.h5`
- **مسیر خروجی (dest_path)**: نتیجه این فاز باید حتماً در پوشه `phase_1_results` ذخیره شود (مثلاً `( phase_1_results/denoised_car.jpeg`

پس از اجرای تابع، به پوشه `phase_1_results` بروید و تصویر خروجی را بررسی کنید. تصویر شما باید از نویزهای نقطه‌ای کاملاً پاک شده باشد، اما **همچنان تار خواهد بود**. این خروجی، ورودی مستقیم شما برای فاز دوم است!

DEblur

فاز دوم: رفع تاری عمیق (Deep Deblurring) و انتخاب معماری

در فاز اول موفق شدید نویزهای فلفل نمکی را از تصویر پاک کنید. اکنون تصویر شما عاری از نویز است، اما همچنان از مشکل **تاری (Blur)** رنج می‌برد و لبه‌های اجسام در آن مشخص نیستند. در این فاز، قصد داریم با استفاده از شبکه‌های عصبی پیشرفته، فرکانس‌های بالا و لبه‌های تیز تصویر را بازیابی کنیم.

در این مرحله، به جای یک مدل واحد، **سه معماری مختلف** در اختیار شما قرار گرفته است. شما آزادی عمل دارید تا هر کدام را که مایل هستید انتخاب کنید، اما توصیه ما استفاده از مدل **ResNet** است.

۱. یک نکته فنی: چرا در این فاز به کدهای کلاس معماری نیاز داریم؟

شاید برایتان سوال پیش آمده باشد که چرا در فاز اول ما فقط فایل مدل را فراخوانی کردیم و شبکه کار کرد، اما در نوت‌بوک این فاز، بلوک‌های کدی طولانی برای کلاس‌های معماری (مثل `class ResNet` یا `class UNet`) وجود دارد؟ دلیل این موضوع، تفاوت در نحوه ذخیره‌سازی مدل‌ها در فریم‌ورک‌های مختلف است:

- **مدل‌های TensorFlow/Keras (فرمت h5 . در فاز اول):** این فرمت معمولاً هم «نقشه ساختاری شبکه» (تعداد لایه‌ها و نحوه اتصال آن‌ها) و هم «وزن‌های آموزش‌دیده» را به صورت یکپارچه درون یک فایل ذخیره می‌کند. در نتیجه، تنها با یک خط کد، کل شبکه ساخته و آماده می‌شود.
- **مدل‌های PyTorch (فرمت .pth . در فاز دوم):** استاندارد پای‌تورچ برای بهینه‌سازی حجم و سرعت، معمولاً به این شکل است که تنها «مقادیر عددی پارامترها و وزن‌ها» (معروف به `state_dict`) را در فایل ذخیره می‌کند. این فایل هیچ ایده‌ای ندارد که این مقادیر متعلق به چه لایه‌هایی هستند! بنابراین، شما **باید** ابتدا اسکلت خالی شبکه را از طریق اجرای کدهای کلاس (مثل `model = ResNet()`) بسازید و سپس وزن‌های فایل `.pth` را مانند یک لباس روی این اسکلت بپوشانید.

به همین دلیل، شما باید حتماً سلول‌های مربوط به ساختار مدل انتخابی خود را در نوت‌بوک اجرا (یا Import) کنید تا تابع `apply_deblur_model` بتواند شبکه را با موفقیت بارگذاری کند.

۲. آشنایی با معماری‌های موجود (مفاهیم پایه)

برای اینکه بتوانید مانند یک مهندس بینایی ماشین تصمیم‌گیری کنید، باید بدانید زیر کلاهی هر یک از این مدل‌ها چه می‌گذرد:

- **معماری اول: CNN (شبکه عصبی پیچشی پایه)**

این شبکه از توالی لایه‌های کانولوشن ساده تشکیل شده است. ترفند اصلی این مدل استفاده از **یادگیری باقیمانده سراسری (Global Residual Learning)** است. این یعنی شبکه به جای اینکه سعی کند کل تصویر را از صفر نقاشی کند، تنها تلاش می‌کند "الگوی تار" را یاد بگیرد و سپس آن را به تصویر تار ورودی اضافه کند تا اثر آن خنثی شود.

- **معماری دوم: ResNet (شبکه باقیمانده عمیق - 🌟 پیشنهاد ما)**

در شبکه‌های عصبی، هرچه لایه‌ها عمیق‌تر شوند، آموزش آن‌ها سخت‌تر می‌شود. مدل ResNet این مشکل را با ایجاد "**مسیرهای میانبر (Skip Connections)**" حل می‌کند. این شبکه به شکل ساعت‌شنی عمل کرده و ابتدا ابعاد تصویر را فشرده می‌کند تا مفهوم تار را درک کند و سپس آن را بازسازی می‌کند. به دلیل وجود میانبرهای محلی، این شبکه معمولاً شارپ‌ترین لبه‌ها را بدون از دست دادن اطلاعات رنگی تولید می‌کند.

- **معماری سوم: U-Net (شبکه U شکل با اتصالات پرشی)**

این معماری که در ابتدا برای تصاویر پزشکی اختراع شد، ساختاری شبیه به حرف "U" دارد. در نیمه اول (رمزگذار)، تصویر دائماً کوچک و فشرده می‌شود و در نیمه دوم (رمزگشا)، دوباره بزرگ می‌شود. قدرت شگفت‌انگیز U-Net در این است که نقشه‌های ویژگی باکیفیت را از نیمه اول، مستقیماً کپی کرده و به نیمه دوم می‌چسباند تا شبکه فراموش نکند که لبه‌های دقیق تصویر پیش از فشرده‌سازی در کجا قرار داشتند.

۳. دانلود وزن‌های مدل (Model Weights)

بسته به اینکه کدام معماری را انتخاب می‌کنید، فایل وزن‌های مربوط به آن را از پلتفرم Kaggle دانلود کرده و در پوشه `models` قرار دهید:

- [دانلود فایل وزن‌های معماری CNN \(فایل CNN.pth\)](#)
- [دانلود فایل وزن‌های معماری ResNet \(فایل ResNet.pth\)](#)
- [دانلود فایل وزن‌های معماری U-Net \(فایل UNet.pth\)](#)

۴. نحوه استفاده از کد (تابع `apply_deblur_model`)

در نوت‌بوک پروژه، کدهای معماری و تابع `apply_deblur_model` برای شما آماده شده است. **ابتدا مطمئن شوید سلول مربوط به کلاس معماری انتخابی خود را اجرا کرده‌اید**، سپس تابع را با آرگومان‌های صحیح فراخوانی کنید:

- `model_path` : مسیر فایل `.pth` که دانلود کردید.
- `input_path` : این مسیر باید دقیقاً به خروجی فاز اول شما (تصویر بدون نویز) اشاره کند (مثلاً `phase_1_results/denoised_image.jpg`).
- `destination_path` : مسیر ذخیره خروجی نهایی در پوشه `phase_2_results`.
- `model_name` : نام معماری انتخابی به صورت رشته متنی (`"cnn"` , `"resnet"` , یا `"unet"`).

نمونه فراخوانی برای مدل ResNet:

Python

```
# اطمینان حاصل کنید که کدهای کلاس ResNet در سلول‌های قبلی اجرا شده‌اند
apply_deblur_model(
    model_path="models/ResNet.pth",
    input_path="phase_1_results/denoised_image.jpg",
    destination_path="phase_2_results/deblurred_image.jpg",
    model_name="resnet"
)
```

پس از اجرای موفق این تابع، به پوشه `phase_2_results` بروید. تصویر شما اکنون باید واضح، شارپ و آماده برای ارزیابی و تحویل به شبکه دسته‌بند (Classifier) باشد!

classification

فاز سوم: آموزش شبکه دسته‌بند و ارزیابی نهایی (Classification)

اکنون به جذاب‌ترین و نهایی‌ترین بخش پروژه رسیده‌ایم! شما در فازهای قبل توانستید با موفقیت تصویر را از نویز و تاری پاک‌سازی کنید. اما آیا این پاک‌سازی به قدری خوب بوده است که یک هوش مصنوعی بتواند محتوای آن تصویر را تشخیص دهد؟

برای پاسخ به این سوال، در این فاز قصد داریم یک شبکه عصبی دسته‌بند (Classifier) را از صفر آموزش دهیم و سپس تصویر بازیابی‌شده‌ی خود را به عنوان آزمون نهایی به آن بسپاریم.

این فاز شامل سه مرحله اجرایی است:

مرحله اول: آموزش مدل روی مجموعه‌داده CIFAR-10

در این مرحله، شما تابع `train_model` را اجرا خواهید کرد. این تابع یک شبکه عصبی با معماری **PreActResNet18** را روی مجموعه‌داده معروف CIFAR-10 آموزش می‌دهد.

آشنایی با CIFAR-10:

این مجموعه‌داده یکی از استانداردترین دیتاست‌های بینایی ماشین است که شامل ۶۰,۰۰۰ تصویر رنگی در ابعاد ۳۲×۳۲ پیکسل می‌باشد. این تصاویر در ۱۰ کلاس مختلف دسته‌بندی شده‌اند: هواپیما، ماشین، پرنده، گربه، گوزن، سگ، قورباغه، اسب، کشتی و کامیون.

⚠ توجه بسیار مهم (پردازش گرافیکی):

آموزش یک شبکه عصبی از صفر، عملیاتی به شدت سنگین و نیازمند محاسبات ماتریسی عظیم است. اگر از سیستم شخصی استفاده می‌کنید، حتماً مطمئن شوید که PyTorch به کارت گرافیک (GPU) شما متصل است. اگر سیستم قدرتمندی ندارید، این مرحله روی CPU ساعت‌ها طول خواهد کشید! در این صورت، حتماً نوت‌بوک خود را در **Google Colab** باز کرده و از منوی Runtime گزینه Change runtime type را روی GPU T4 تنظیم کنید تا آموزش در چند دقیقه به پایان برسد.

یک نکته فنی (تفاوت در ذخیره‌سازی مدل):

در فاز دوم به یاد دارید که گفتیم PyTorch فقط وزن‌ها را ذخیره می‌کند و شما باید ساختار کلاس (مثل ResNet) را خودتان بسازید؟ در این مرحله ما از ابزاری به نام **TorchScript** (دستور `torch.jit.trace`) استفاده کرده‌ایم. این ابزار قدرتمند، هم «معماری شبکه» و هم «وزن‌ها» را با هم ترکیب کرده و به صورت یک گراف اجرایی واحد در فایلی به نام `preact_resnet18_standalone.pt` ذخیره می‌کند. بنابراین برای استفاده از این مدل در مراحل بعدی، دیگر نیازی به اجرای کدهای کلاس معماری نخواهیم داشت!

نحوه اجرا:

کافیست سلول‌های مربوط به معماری `PreActResNet18` را اجرا کرده و سپس تابع آموزش را فراخوانی کنید:

Python

```
# آموزش مدل و ذخیره آن  
train_model(model_architect=PreActResNet18)
```

پس از اتمام آموزش، فایل مدل در پوشه `models/preact_resnet18_standalone.pt` ذخیره شده و نمودار روند آموزش (Loss و Accuracy) نیز تولید می‌شود.

مرحله دوم: ارزیابی مدل با ماتریس درهم‌ریختگی (Confusion Matrix)

پیش از آنکه تصویر اصلی خودمان را به مدل بدهیم، باید بدانیم مدلی که آموزش داده‌ایم چقدر باهوش است و نقاط ضعف آن کجاست. برای این کار از تابع `plot_cifar_confusion_matrix` استفاده می‌کنیم.

ماتریس درهم‌ریختگی چیست؟

ماتریس درهم‌ریختگی یک جدول بصری است که عملکرد الگوریتم را روی ۱۰,۰۰۰ تصویر تست نشان می‌دهد.

- **سطرها:** نشان‌دهنده کلاس واقعی تصویر هستند.
- **ستون‌ها:** نشان‌دهنده کلاسی هستند که شبکه ما پیش‌بینی کرده است.

- **قطر اصلی:** اعدادی که روی قطر اصلی ماتریس قرار می‌گیرند، نشان‌دهنده پیش‌بینی‌های کاملاً درست هستند. هرچه این اعداد بزرگتر و رنگ خانه‌هایشان تیره‌تر باشد، مدل بهتر عمل کرده است. اعدادی که خارج از قطر اصلی قرار دارند، نشان‌دهنده **اشتباهات (Confusion)** شبکه هستند (مثلاً شبکه چند بار تصویر «سگ» را به اشتباه «گربه» تشخیص داده است).

نحوه اجرا:

Python

```
plot_cifar_confusion_matrix(  
    model_path="models/preact_resnet18_standalone.pt",  
    data_dir="./data"  
)
```

با اجرای این کد، ماتریس تولید شده و در قالب یک تصویر در فایل‌های شما ذخیره می‌شود. حتماً آن را تحلیل کنید!

مرحله سوم: آزمون نهایی خط لوله (Final Classification)

لحظه حقیقت فرا رسید! اکنون می‌خواهیم تصویر مخدوشی که در فاز ۱ (حذف نویز) و فاز ۲ (رفع تاری) پردازش کردیم را به این شبکه دسته‌بندی جدید بدهیم تا ببینیم آیا عملیات بازیابی ما موفقیت‌آمیز بوده است یا خیر. شما باید از تابع `classifier` استفاده کنید و تصویر خروجی فاز دوم را به آن پاس دهید.

نحوه اجرا:

Python

```
classifier(  
    model_path="models/preact_resnet18_standalone.pt",  
    image_path="phase_2_results/deblurred_image.jpg", # خروجی فاز دوم  
    ground_truth="car" # کلاس واقعی تصویر خود را به انگلیسی بنویسید (مثلاً car, plane, bird)  
)
```

خروجی مورد انتظار:

این تابع در کنسول به شما نشان خواهد داد که مدل با چه درصد اطمینانی (Confidence) تصویر بازیابی شده شما را تشخیص داده است. اگر پیش‌بینی مدل (Predicted Class) با کلاس واقعی (Ground Truth) یکسان باشد، به این معناست که خط لوله پردازش تصویر شما با موفقیت توانسته است ویژگی‌های معنایی یک تصویر کاملاً مخدوش را برای ماشین قابل درک کند!

project delivery

مرحله نهایی: مفاهیم کلیدی، تحویل پروژه و گزارش تحلیلی

به پایان این ماجراجویی در دنیای یادگیری عمیق رسیدید! از آنجایی که ممکن است برخی از شما هنوز دروس تخصصی یادگیری ماشین را نگذرانده باشید، پیش از آنکه از شما بخواهیم نتایج خود را تحلیل کنید، یک درسنامه کوتاه و ساده از مفاهیمی که در این پروژه استفاده کردید را برایتان آماده کرده‌ایم.

ابتدا این درسنامه را با دقت بخوانید و سپس بر اساس آن، به سوالات انتهای صفحه پاسخ دهید.

بخش اول: درسنامه کوتاه (Crash Course) مفاهیم یادگیری عمیق

۱. چرا مقادیر پیکسل‌ها را بین ۰ تا ۱ نرمال کردیم؟

شبکه‌های عصبی در واقع مجموعه‌ای از معادلات ریاضی و ضرب ماتریس‌ها هستند. اگر مقادیر ورودی بزرگ باشند (مثل پیکسل‌های سفید با مقدار ۲۵۵)، وقتی این اعداد در طول لایه‌های شبکه در هم ضرب می‌شوند، به سرعت به اعداد غول‌پیکر و غیرقابل کنترلی تبدیل می‌شوند (مشکل انفجار محاسباتی). با تقسیم پیکسل‌ها بر ۲۵۵ و تبدیل آن‌ها به بازه $[0.0, 1.0]$ ، ریاضیات شبکه پایدار و آرام باقی می‌ماند و مدل می‌تواند به راحتی آموزش ببیند.

۲. جادوی «اتصالات میانبر» (Skip Connections) در ResNet چیست؟

در شبکه‌های ساده (مثل CNN پایه)، وقتی تصویر از لایه‌های مختلف عبور می‌کند تا پردازش شود، فشرده شده و جزئیات دقیق و لبه‌های تیز خود را فراموش می‌کند. معماری‌هایی مثل ResNet و U-Net دارای مسیرهایی به نام «میانبر» هستند. این میانبرها مانند یک بزرگراه عمل می‌کنند و جزئیات باکیفیت تصویر اصلی را مستقیماً به انتهای شبکه می‌فرستند تا در زمان بازسازی تصویر، لبه‌ها تار نشوند و شارپ باقی بمانند.

۳. نمودار Loss (خطا) به ما چه می‌گوید؟ (بیش‌برازش در برابر آموزش صحیح)

در فاز سوم، شما نموداری از روند آموزش (Train Loss) و ارزیابی (Val Loss) تولید کردید.

- **آموزش صحیح (Good Fit):** اگر هم خطای Train و هم خطای Val با هم در حال کاهش باشند، یعنی مدل شما واقعاً در حال یادگیری مفاهیم است.
- **بیش‌برازش (Overfitting):** اگر خطای Train به شدت کاهش یابد (به صفر نزدیک شود) اما خطای Val ثابت بماند یا **بالا برود**، یعنی فاجعه رخ داده است! مدل شما مفاهیم را یاد نگرفته، بلکه فقط داده‌های آموزشی را "حفظ کرده" است (مثل دانش‌آموزی که به جای یادگیری فرمول، جواب سوالات نمونه‌سوال را حفظ می‌کند و در امتحان اصلی مردود می‌شود).

۴. تفاوت Accuracy (دقت) و F1-Score چیست؟

- به سادگی می‌گویند چند درصد از کل تصاویر درست تشخیص داده شده‌اند. اگر **Accuracy (دقت)** دیتاست شما متعادل باشد (مثلاً تعداد سگ‌ها و گربه‌ها برابر باشد) معیار خوبی است. اما فرض کنید در دیتاستی ۹۹ عکس سالم و فقط ۱ عکس سرطانی داریم. اگر مدل همیشه بگوید "سالم"، دقت او ۹۹٪ است! اما در واقع مدل کاملاً بی‌فایده است چون آن ۱ عکس سرطانی را پیدا نکرده است.
- **F1-Score:** این معیار، ترکیبی ریاضی است که روی "تعادل" تمرکز دارد. این معیار گول دیتاست‌های نامتعادل را نمی‌خورد و فقط زمانی بالا می‌رود که مدل در تشخیص **همه کلاس‌ها** (حتی کلاس‌های کمیاب) خوب عمل کند.

بخش دوم: فایل‌های مورد نیاز برای ارسال (Uploads)

لطفاً تمام فایل‌های زیر را در یک فایل فشرده (ZIP) با فرمت `studentName_StudentNumber_Project.zip` قرار داده و در سامانه Quera بارگذاری کنید:

۱. **نوت‌بوک نهایی:** فایل `ipynb`. شما که تمامی سلول‌های آن اجرا شده و خروجی‌های آن قابل مشاهده است.

۲. **تصاویر بازیابی‌شده:** فایل‌های `phase1_denoised.jpg` و `phase2_deblurred.jpg`.

۳. **نمودارهای استخراج‌شده:** نمودار آموزش (`PreActResNet_training_metrics.png`) و ماتریس درهم‌ریختگی (`cifar10_confusion_matrix.png`).

۴. **گزارش نهایی (PDF):** یک فایل خوانا شامل پاسخ به «سوالات تحلیلی» زیر.

بخش سوم: سوالات تحلیلی (گزارش پروژه)

لطفاً در فایل PDF خود، با استفاده از مفاهیمی که در درسنامه بالا آموختید و نتایجی که در اجرای کدهایتان گرفتید، به این سوالات پاسخ دهید:

۱. **تحلیل پیش‌پردازش:** با توجه به مفهوم شماره ۱ در درسنامه، توضیح دهید که اگر در کد فاز اول، خط مربوط به `input_tensor = image.astype(np.float32) / 255.0` را حذف می‌کردیم، چه اتفاقی برای فرآیند پیش‌بینی مدل می‌افتاد و چرا؟

۲. **تحلیل معماری رفع تار:** در فاز دوم، شما مقدار PSNR را برای تصویر خروجی محاسبه کردید. با توجه به مفهوم شماره ۲ در درسنامه (جادوی اتصالات میانبر)، توضیح دهید که چرا شبکه‌ای مانند ResNet در رفع تار لبه‌ها بهتر از یک شبکه ساده بدون میانبر عمل می‌کند؟

۳. **تحلیل نمودار یادگیری (Learning Curves):** به نمودار ذخیره‌شده از فاز سوم خودتان (Loss) نگاه کنید. با توجه به مفهوم شماره ۳، آیا مدل شما دچار بیش‌برازش (Overfitting) شده است یا آموزش درستی داشته است؟ دلیل خود را با توجه به رفتار خطوط Train و Val در نمودارتان بنویسید.

۴. **درک معیارهای ارزیابی:** با توجه به مفهوم شماره ۴ در درسنامه، یک مثال واقعی (به جز مثال تشخیص سرطان) بزنید که در آن دیتاست به شدت نامتعادل است و استفاده از معیار Accuracy ما را فریب می‌دهد، در حالی که باید حتماً از معیار F1-Score استفاده کنیم.

۵. **تحلیل ماتریس درهم‌ریختگی (Confusion Matrix):** به تصویر ماتریس درهم‌ریختگی خودتان نگاه کنید.

- الف) مدل شما کدام دو کلاس را بیشتر از همه با یکدیگر اشتباه گرفته است؟ (بیشترین عدد خارج از قطر اصلی).
- ب) شبکه‌های کانولوشنی (CNN) برای تشخیص، به ویژگی‌های بصری مانند بافت (Texture)، شکل (Shape) و رنگ پس‌زمینه نگاه می‌کنند. با توجه به این موضوع، حدس می‌زنید چرا شبکه در تفکیک این دو کلاس خاص دچار اشتباه شده است؟ (مثلاً چه ویژگی مشترکی بین آن‌هاست که شبکه را فریب داده است؟)

پایان پروژه - از تلاش شما سپاسگزاریم! 🚀

